

torch.autograd.functional.hvp#

<https://pytorch.org/docs/stable/generated/torch.autograd.functional.hvp.html>

Description:

Compute the dot product between the scalar function's Hessian and a vector v at a specified point. `func` (function) – a Python function that takes Tensor inputs and returns a Tensor with a single element. `inputs` (tuple of Tensors or Tensor) – inputs to the function `func`. `v` (tuple of Tensors or Tensor) – The vector for which the Hessian vector product is computed. Must be the same size as the input of `func`. This argument is optional when `func`'s input contains a single element and (if it is not provided) will be set as a Tensor containing a single 1. `create_graph` (bool, optional) – If True, both the output and result will be computed in a differentiable way. Note that when `strict` is False, the result can not require gradients or be disconnected from the inputs. Defaults to False.

Function Signature

```
torch.autograd.functional.hvp(func, inputs, v=None,  
create_graph=False, strict=False)[source]
```

Parameters

Returns

tuple with:

Return type

Returns:

output (tuple)

Code Examples

```
>>> def pow_reducer(x):
...     return x.pow(3).sum()
>>> inputs = torch.rand(2, 2)
>>> v = torch.ones(2, 2)
>>> hvp(pow_reducer, inputs, v)
(tensor(0.1448),
 tensor([[2.0239, 1.6456],
         [2.4988, 1.4310]]))
```

```
>>> hvp(pow_reducer, inputs, v, create_graph=True)
(tensor(0.1448, grad_fn=<SumBackward0>),
 tensor([[2.0239, 1.6456],
         [2.4988, 1.4310]], grad_fn=<MulBackward0>))
```

```
>>> def pow_adder_reducer(x, y):
...     return (2 * x.pow(2) + 3 * y.pow(2)).sum()
>>> inputs = (torch.rand(2), torch.rand(2))
>>> v = (torch.zeros(2), torch.ones(2))
>>> hvp(pow_adder_reducer, inputs, v)
(tensor(2.3030),
 (tensor([0., 0.]),
  tensor([6., 6.])))
```

Notes & Warnings

NOTE

Note This function is significantly slower than vhp due to backward mode AD constraints. If your functions is twice continuously differentiable, then $\text{hvp} = \text{vhp.t}()$. So if you know that your function satisfies this condition, you should use vhp instead that is much faster with the current implementation.

Detailed Documentation

Documentation

Compute the dot product between the scalar function's Hessian and a vector v at a specified point.

func (function) – a Python function that takes Tensor inputs and returns a Tensor with a single element.

inputs (tuple of Tensors or Tensor) – inputs to the function func.

v (tuple of Tensors or Tensor) – The vector for which the Hessian vector product is computed. Must be the same size as the input of func. This argument is optional when func's input contains a single element and (if it is not provided) will be set as a Tensor containing a single 1.

create_graph (bool, optional) – If True, both the output and result will be computed in a differentiable way. Note that when strict is False, the result can not require gradients or

be disconnected from the inputs. Defaults to False.

`strict` (bool, optional) – If True, an error will be raised when we detect that there exists an input such that all the outputs are independent of it. If False, we return a Tensor of zeros as the hvp for said inputs, which is the expected mathematical value. Defaults to False.

`func_output` (tuple of Tensors or Tensor): output of `func(inputs)`

`hvp` (tuple of Tensors or Tensor): result of the dot product with the same shape as the inputs.

This function is significantly slower than `vhp` due to backward mode AD constraints. If your function is twice continuously differentiable, then `hvp = vhp.t()`. So if you know that your function satisfies this condition, you should use `vhp` instead that is much faster with the current implementation.